Member-only story

Decision Trees: A Complete Introduction

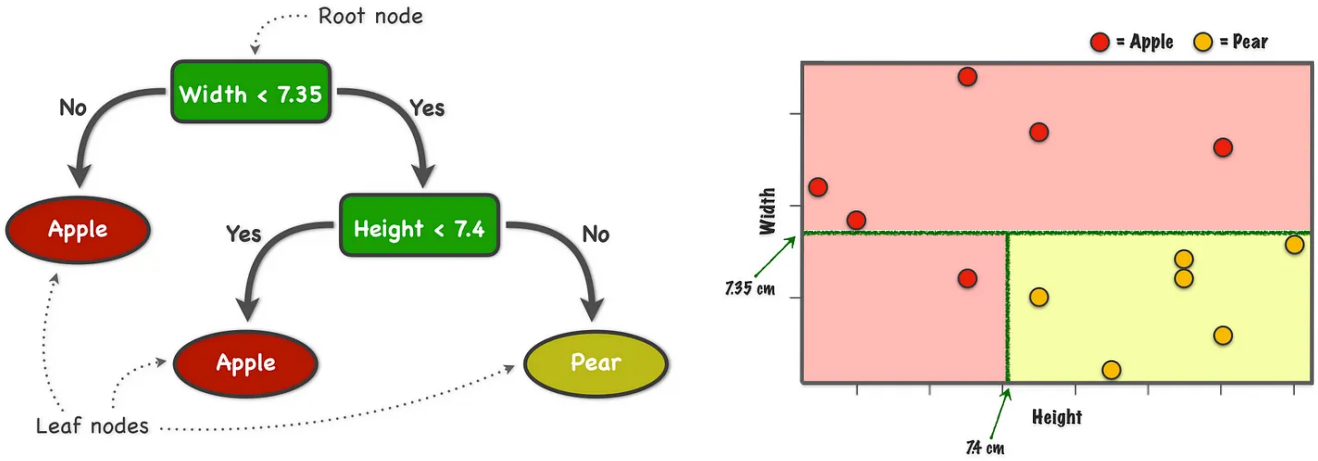
Alan Jeffares · Following
Published in Towards Data Science
13 min read · Jul 25, 2018

Decision tree's are one of many supervised learning algorithms available to anyone looking to make predictions of future events based on some historical data and, although there is no one generic tool optimal for all problems, decision tree's are hugely popular and turn out to be very effective in many machine learning applications.

To understand the intuition behind the decision tree, consider the problem of designing an algorithm to automatically differentiate between apples and pears (*class labels*) given only their width and height measurements (*features*).

Width	Height	Fruit
7.1	7.3	Apple
7.9	7.5	Apple
7.4	7.0	Apple
8.2	7.3	Apple
7.6	6.9	Apple
7.8	8.0	Apple
7.0	7.5	Pear
7.1	7.9	Pear
6.8	8.0	Pear
6.6	7.7	Pear
7.3	8.2	Pear
7.2	7.9	Pear

Apples tend to be short and fat while pears are usually taller and more slight. Based on this knowledge we can ask a series of questions which eventually lead to an educated guess at the true class of the mysterious item of fruit. For example, to classify a piece of fruit we might first ask if the width is less than 7.35 cm and then we could ask if the height is less than 7.4 cm. If the answer to both of these questions is *yes* then we can conclude (with a high degree of confidence) that the unlabelled item is an apple. The process we have just described is, in fact, exactly that of a decision tree.



A visual representation of a decision tree and how this translates into 2D feature space

The analogy of a (upside down) tree makes sense when we look at how all of our questions or *splits* stem from a root question (*root node*) and sequentially branch out to eventually reach some terminal decision (*leaf node*). Beside the tree, we can see exactly how this translates onto the two dimensional feature space. The splits divide the plane into a number of boxes with each one corresponding to a classification (apple/pear) for an observation. This really simple concept does a pretty good job of creating an automatic system for labelling fruit and modern computers can easily adopt this approach to classify thousands of observations per second.

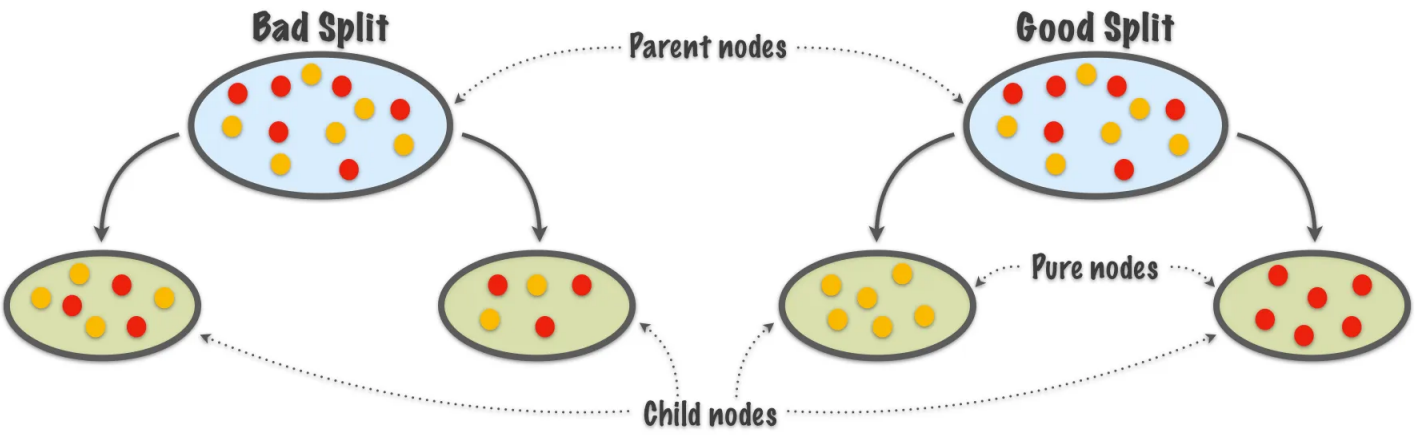
There's just one thing missing in what has been discussed so far: How do we build a decision tree? Trial and error would become a pretty laborious task as we get more data with increasing complexity and, even if we take this approach, there's a huge number of trees we could potentially make, how do we design one that is optimal? The next section discusses the process of building decision trees from labelled data.

Building Decision Trees

Given a set of labelled data (*training data*) we wish to build a decision tree that will make accurate predictions on both the training data and on any new unseen observations. Depending on the data in question, decision trees may require more

splits than the one in the previous example but the concept is always the same: Make a series of good splits that correctly classify observations as their true class label. So how do we define what would make *good splits*?

Well, at each split we wish to find a feature that divides the labelled data in such a way as to have the child nodes be more homogeneous than the parent node that they came from. In other words we wish the nodes to become more *pure* as we move down the tree and we wish to choose our splits to maximise this increase in node purity.



There are two popular choices for measuring this concept of node purity numerically, but for now we will focus on entropy (with its alternative, Gini impurity, covered later on). More concretely, for observations $\{x_1, x_2, \dots, xn\} \in X$ where $P(x_i)$ is the relative frequency of observation *i*, then entropy is defined as:

$$H(X) = -\sum_{i=1}^{n} P(x_i) \log_2 P(x_i)$$

This equation gives low values for purer nodes, thus we wish to choose the split that minimises entropy. This seems to make sense, but there is a problem: This set up can be abused by certain choices of splits of a parent node such that one large child node contains practically all of the observations while a second tiny child node is pure. This adds another split to the tree that doesn't really improve the overall performance (as it only accounts for one extreme value) while also adding

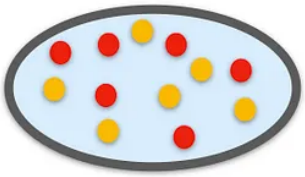
unnecessary depth and complexity to the tree, especially if this happens repeatedly in the same tree.

The solution? Information gain. This is a popular information theoretic approach that compares the entropy of the parent node before the split to that of a weighted sum of the child nodes after the split where the weights are proportional to the number of observations in each node. This effectively penalises small, pure nodes based on their size just as we required. More formally, for a feature F and a set of observations X, information gain is defined as:

$$\begin{aligned} IG(X,F) &= (original\ entropy) - (entropy\ after\ the\ split) \\ IG(X,F) &= H(X) - \sum_{i=1}^{n} \frac{|x_i|}{X} H(x_i) \end{aligned}$$

Since information gain subtracts the entropy of the child nodes our choice of split will be the one that maximises the information gain. This all becomes more clear by working through the example displayed in the previous figure and comparing the information gain values we obtain from two different proposed splits on the same root node.

Calculate Entropy



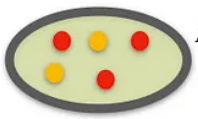
$$H(X) = \sum_{i=1}^{n} P(x_i) log_2 P(x_i)$$

$$= -\frac{6}{12} \cdot log_2(\frac{6}{12}) - \frac{6}{12} \cdot log_2(\frac{6}{12})$$

$$= \frac{1}{2} + \frac{1}{2} = 1$$

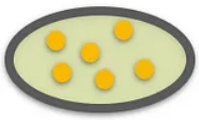
$$H(X) = -\overbrace{\frac{4}{7} \cdot log_2(\frac{4}{7})}^{} - \overbrace{\frac{3}{7} \cdot log_2(\frac{3}{7})}^{}$$

$$= 0.4613 + 0.5239 = 0.9852$$



$$H(X) = -\overbrace{\frac{3}{5} \cdot log_2(\frac{3}{5})}^{} - \overbrace{\frac{2}{5} \cdot log_2(\frac{2}{5})}^{}$$

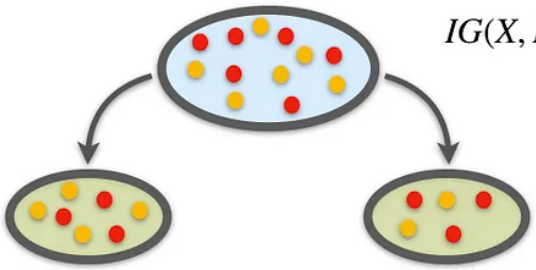
$$= 0.4422 + 0.5288 = 0.9710$$



$$H(X) = -\frac{6}{6} \cdot log_2(\frac{6}{6}) = 0$$

$$H(X) = -\frac{6}{6} \cdot log_2(\frac{6}{6}) = 0$$

Calculate Information Gain

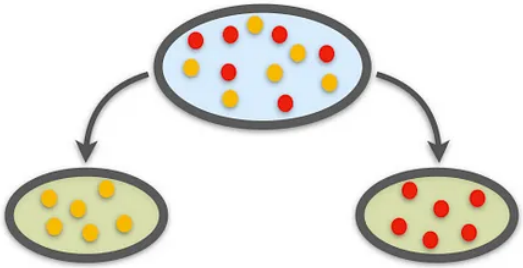


$$IG(X, F) = (original\ entropy) - (entropy\ after\ split)$$

$$= H(X) - \sum_{i=1}^{n} \frac{|x_i|}{X} H(x_i)$$

$$= 1 - \frac{7}{12} \cdot (0.9852) - \frac{5}{12} \cdot (0.9710) = 0.0213$$

$$IG(X, F) = 1 - \frac{6}{12} \cdot (0) - \frac{6}{12} \cdot (0) = 1$$



Notice that the first proposed split only improves the node purity very slightly with

an information gain of just 0.0213. The second split, however, splits the root node into two perfectly pure nodes and this is reflected by the information gain score of 1. Clearly, the better choice of split can be measured by choosing the one that maximises the information gain based on entropy values.

Now there is just one final piece to this puzzle and then we are ready to build our own decision tree from scratch, and that is choosing where to split continuous features. Continuous data (such as height or width) can be split at a potentially infinite number of values whereas categorical data (such as fruit colour or blood type) have a finite number of values to split on and calculate the information gain for. Consider the following two features:

- Fruit Colour (categorical) - *{Green, Red, Yellow, Other}*
- Fruit Height (continuous) - *{5.65, 5.78, 6.34, 6.84, 6.86}*

For fruit colour we can easily check all of the possible splits and select the one that maximises the information gain as described before. When we attempt to take this approach for fruit height there are way too many values to potentially split on. We could use 5.92, 5.93 or even 5.92534… Which should be used? Well the obvious choice is to take the midpoint between each two values, in this case we would have four values to split on and check the information gain for. This doesn't scale very well though. If we had 1000 unique pieces of fruit in the dataset we would have 999 possible splits to check which can become pretty computationally heavy. The solution is to apply a popular clustering algorithm, such as k-means, to split fruit height into a predefined number of groups so our data might look like this:

$$\{\{5.65, 5.78\}, \{6.34\}, \{6.84, 6.86\}\}$$

Now we only need to check a much more manageable number of potential splits (those being 6.06 and 6.59, can you see why?). We have effectively converted our continuous values into discrete intervals and in practise this often manages to obtain the best information gain without wasting time checking additional values unnecessarily.

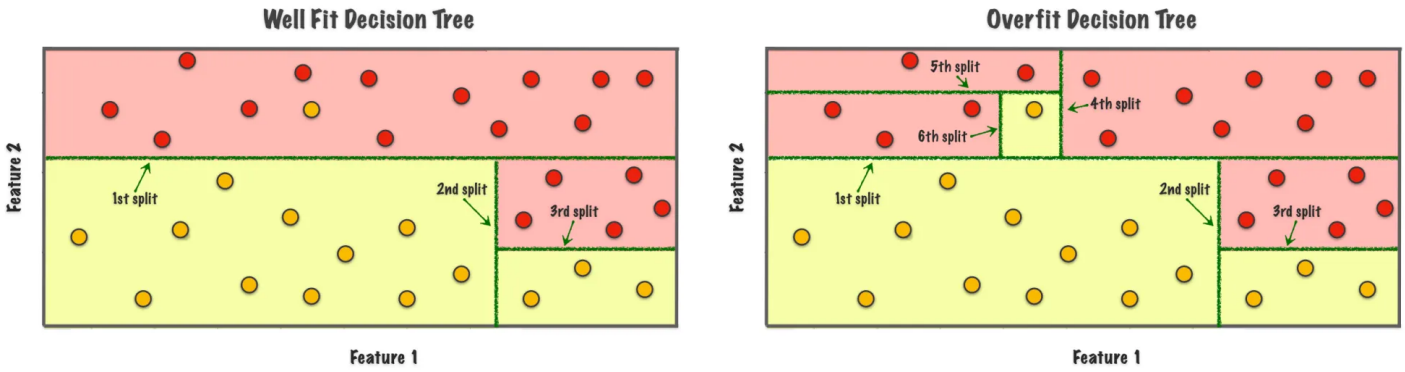
And thats pretty much it! This is all the theory we need to start building some decision trees. We can put all of this together into a decision tree algorithm which is roughly based on the original ID3 implementation. In the following pseudocode `S` refers to a set of data (such as the apple/pear data), `C` refers to the class labels (e.g. apple and pear), `A` refers to features (e.g. height or width) and `v` refers to the possible values a feature is split into (e.g. height < 7.4 cm).

```
ID3(S):
IF all examples in S belong to class C:
    - return new leaf node and label with class C
ELSE:
    - select a feature A based on some feature selection criterion
    - generate a new tree node with A as the test feature
    - FOR EACH value v[i] of A:
        - let S[i] ⊂ S contain all examples with A = v[i]
        - build subtree by applying ID3(S[i])
```

The next section proceeds by looking at some approaches we can take to improve the performance in practise by preventing a common machine learning pitfall known as overfitting.

Overfitting

We mentioned earlier that a desirable characteristic in our decision trees (and, in fact, in all classification problems) is good generalisation. This means that we wish for the model fit on the labelled training data to make predictions that are just as accurate on new unseen observations. When this does not happen it is often due to a phenomena known as overfitting. In decision trees this occurs when the splits in the tree are too specifically defined for the training data. Below we observe a decision tree that has added three consecutive splits in order to classify a single outlier, in practise these additional splits will not improve performance on new data and may in fact worsen it.



Adding three new splits to capture a single data point is probably overfitting

There are three approaches to preventing overfitting in decision trees:

- Early stopping — Build the decision tree while applying some criterion that stops the decision trees growth before it overfits to the training data.
- Pruning — Build the decision tree and allow it to overfit to the training data, then prune it back to remove the elements causing overfitting.
- Data preprocessing — Making some changes to the initial data before ever building the tree

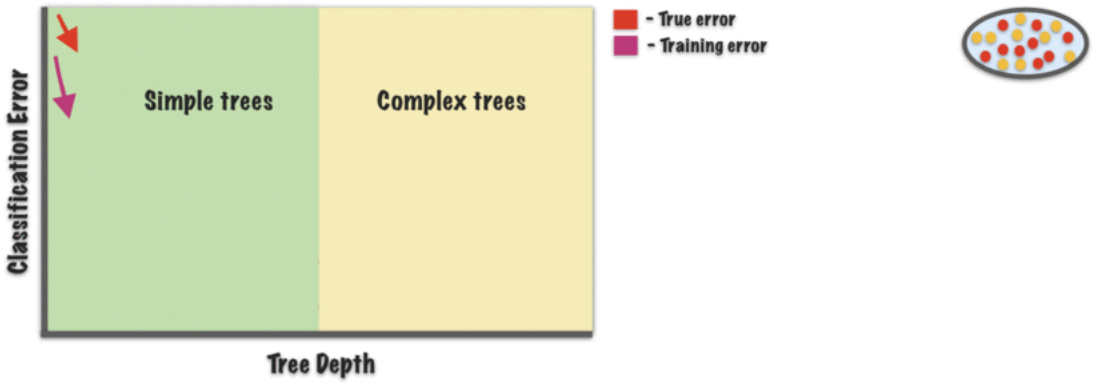
We will discuss the first two of these approaches here.

Early stopping (or pre-pruning) is where we stop the growth of a decision tree early before it overfits to the training data. The idea here is to stop the trees growth before it makes overly niche splits that don't generalise well and, in practise, this is often used. Recall that in the previous section the root node sequentially added splits until the child nodes were pure, now we need an alternative rule that tells the tree to stop growing before the nodes are pure and classifies the new terminal nodes by their majority class. There are a huge number of potential stopping rules we could conceive of, so here is a non-exhaustive list of some popular choices:

- Maximum tree depth — Simply pre define an arbitrary number for the maximum depth (or max number of splits) and once the tree reaches this value

the growing process terminates.

- Minimum number in node — Define a minimum number of observations to appear in any child node for a split to be valid.
- Minimum decrease in impurity — Define a minimum acceptable decrease in impurity for a split to be accepted.
- Maximum features — Not strictly speaking a stopping rule but only considering a subset of the available features to split on may improves the final trees generalisability.

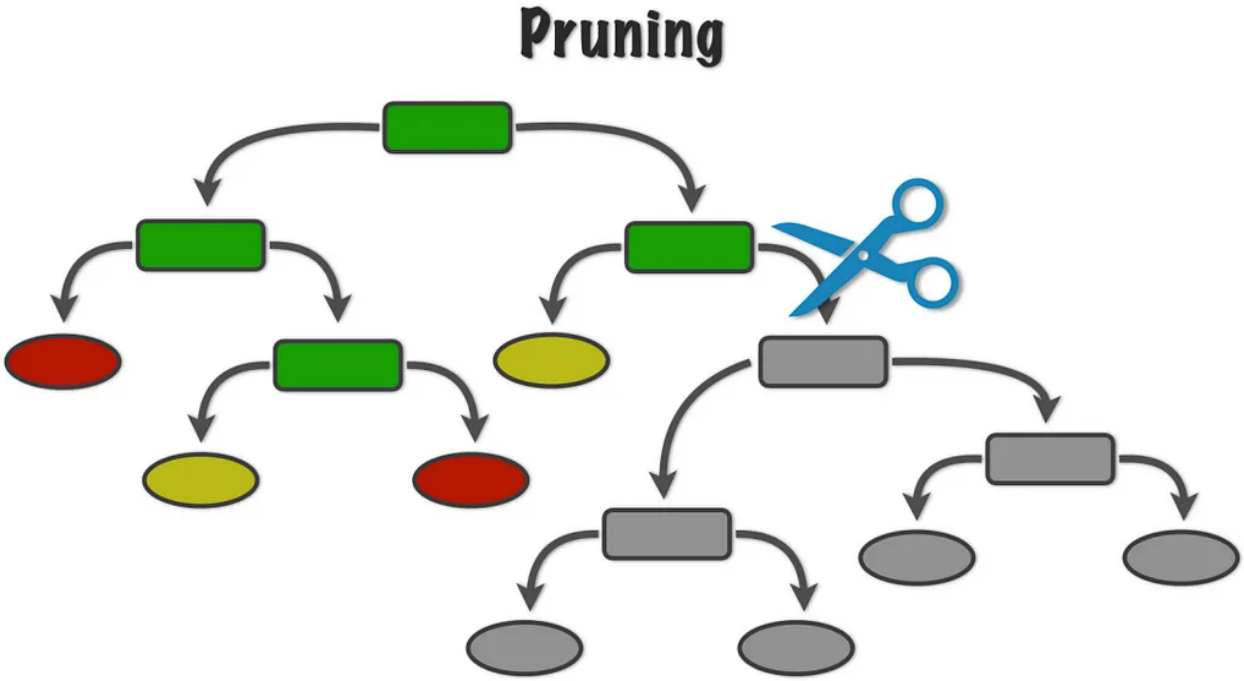


We wish to find a tree that will generalise well on the population

Pruning (or post-pruning) on the other hand takes a tree that has already been overfit and makes some adjustments to reduce/remove the observed overfitting. A good pruning rule will pinpoint the splits that don't generalise well, often by using an independent test set, and remove them from the tree. Again, there are many different approaches to implementing pruning but three popular choices are:

- Critical value pruning — Retrospectively estimates the strength of each node from calculations done in the tree building stage. Nodes that don't achieve a certain critical value are pruned, unless a node further along the branch does reach it.
- Error complexity pruning — Generates a series of trees each made by pruning the full tree by different amounts and selects one of these by assessing its performance with an independent data set.

- Reduced error pruning — Runs the independent test data through the full tree and, for each non-leaf node, compares the number of errors if the sub tree from that node is kept vs removed. The pruned node will often make fewer errors using the new test data than the sub tree makes. The node that sees the biggest difference in performance is pruned and this process is continued until further pruning will increase the misclassification rate.

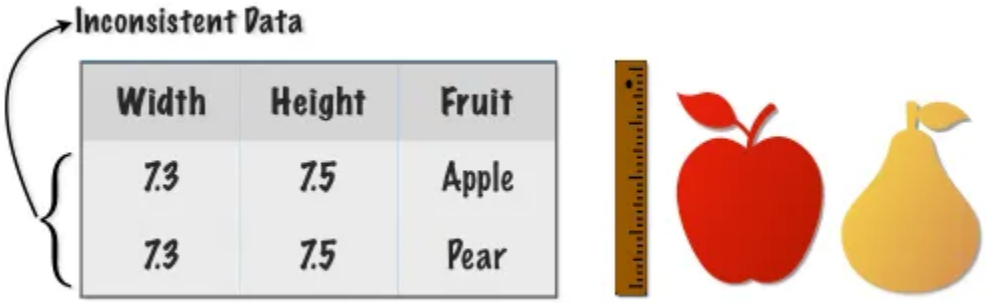


When it comes to using a decision tree in practise there is no one size fits all solution to overfitting. Designing the optimal decision tree often requires trial and error as well as some domain knowledge and, as such, a number of the above approaches should be experimented with. This is a process known as *hyperparameter optimisation*.

Further considerations

Whats been discussed up to this point covers the main theory behind the decision tree classifier leaving this section to (hopefully) answer any remaining questions or considerations we might have.

Handling Inconsistent Data — In the real world, data sets are not always as tidy as we would like and inconsistent data is an example of this. Inconsistent data occurs when two identical observations have different class labels, in our data this might occur if two fruit had identical height and width measurements but different labels due to one being a pear and the other an apple. Clearly, in this case, the final leaf node can not be pure as there is no feature we can choose to differentiate between the two fruit. When this occurs, a common approach is to set the class predictions for the impure node in question to be that of the majority class in that node. In other words, if this node contained five apples and just a single pear the decision tree would predict new observations in this node as being labelled apple. However, if the node has equal observations from majority classes then a label may have to be randomly selected.

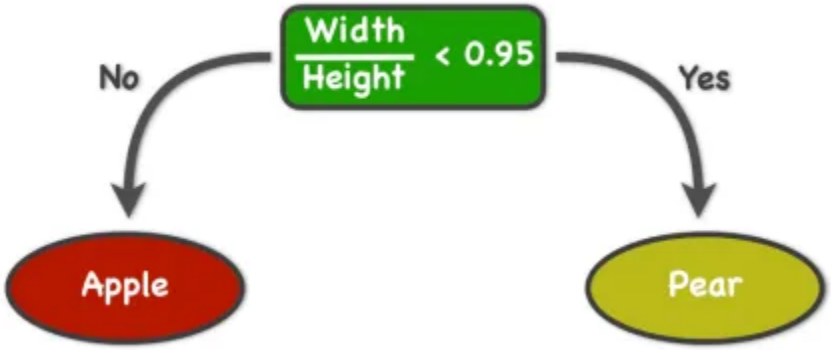


Dealing With Missing Values — A missing value occurs when one or more of an observations features are somehow not included in the data. If our data contained an apple with a width value of 7.5 cm but no height measurement then this would be considered a missing value. Missing value observations are problematic as any missing features cannot be considered for splitting on, and this upsets the entire process of building the decision tree. In early versions of decision trees, observations with missing values were simply thrown out from the building process. More recently, however, sophisticated approaches have emerged that allow information from missing values to be retained.

One such approach is used in R's `rpart` package which ensures that any observation with values for the dependent variable and at least one independent variable will

participate in the modelling. Using this procedure, the information gain formula takes a minor adjustment to allow for observations that contain missing values in the choice of split. Once the split has been defined, the concept of *surrogate variables* is introduced for making predictions. These may be thought of as backup variables that are used in the case that the relevant feature is missing. A ranked list of alternative surrogate variables is created and, if they produce better results than blindly going with the majority class, they are retained. This approach allows us to use observations with missing values to build the decision tree as well as in making predictions.

Oblique Splits — Up until this point we have only considered splitting based on a single feature but it is worth being aware that we could extend this approach to consider using linear combinations of features. These non-orthogonal or oblique splits might be more efficient in certain appropriate scenarios. An example of an oblique split on the apple and pear dataset is included below where using the ratio of height to width has the benefit of correctly classifying all of the observations in a single split. However the benefits of using oblique splits come with some drawbacks, these including reduced interpretability and increased influence of missing values and outliers.



It is also worth noting that the above decision tree is also what is know as a *decision stump*. A decision stump is a decision tree where its root node has just a single split with all nodes created being terminal.

Gini Impurity — Gini impurity is a measurement of the likelihood of an incorrect

classification of a new instance of a random variable, if that new instance were randomly classified according to the distribution of class labels from the data set. It can be defined as:

$$Gini(X) = 1 - \sum_{i=1}^{n} P(x_i)^2$$

In practise there is practically no difference in performance between Gini impurity and Entropy apart from a slight decrease in computational speed with entropy due to its use of the log function.

Regression Trees — Technically, the decision tree that has been discussed in this blog is a *classification tree*. The task has been to identify which class or category an observation belongs to from a finite set of possibilities (e.g. apple or pear). Sometimes, however, our response variable can be numeric rather than categorical (e.g. fruit price) and in this case we are solving a slightly different problem. Note that this only refers to the case where the response variable (that we are trying to predict) is numeric, not the predictor variables (that we are using to make the predictions) as we already know how to deal with numeric predictors. In the case where the response variable is numeric the task becomes a regression problem and in this case we are making what are known as *regression trees*.

Fortunately, regression trees aren't too different to classification trees and the only adjustment we need to make is how we measure impurity. Rather than measuring the impurity of the nodes, we now wish to choose the split that minimises the sum of squared error in the child nodes. Then, when making predictions, we use the mean value (from the training data) of whatever leaf node an observation falls into. More formally, for all candidate splits where *leaves* is the set of all leaves created by a split, y_i is the numeric response value of observation *i* and

$$\bar{y}_c = \frac{1}{n_c} \sum_{i \in c} y_i$$

is the mean numeric response value for a given leaf *c*, we wish to choose the split that minimises the following expression:

$$SSE = \sum_{c \in leaves} \sum_{i \in c} (y_i - \bar{y}_c)^2$$

R implementation

```r
# rpart is a popular implementation of the decision tree classifier
library(rpart)
library(rpart.plot)


# lets load in the fruit data from earlier
fruit.data <- data.frame(
  Width =  c(7.1, 7.9, 7.4, 8.2, 7.6, 7.8, 7, 7.1, 6.8, 6.6, 7.3, 7.2),
  Height = c(7.3, 7.5, 7, 7.3, 6.9, 8, 7.5, 7.9, 8, 7.7, 8.2, 7.9),
  Fruit = rep(c("Apple", "Pear"), each = 6, len = 12)
)


# building the decision tree via the rpart() function
fit <- rpart(formula = Fruit ~ Width + Height, method = "class",
             data = fruit.data, minsplit = 1)
# formula: tells R that Fruit is the response variable being predicted using the features Width
# method: "class" for classification tree, "anova" for regression tree
# data: input the dataframe
# minsplit: minimum number of observations per leaf, default is 20
summary(fit) # provides detailed info on each node, variable importance, ...
rpart.plot(fit) # exactly as described earlier!


# now lets use this tree to make some predictions
predict(fit) # the tree classifies the training data perfectly (as we would expect)
# lets see how it does on some brand new data
new.data <- data.frame(Width = c(8, 6.9), Height = c(7.2, 8.1))
new.data # one is short and fat, the other is slim and tall
predict(fit, new.data, type = 'class') # predicted as Apple and Pear as we would hope


# now lets se how this tree might be pruned
pruned.tree <- prune(fit, cp = 0.5) # this function uses cost-complexity pruning
rpart.plot(pruned.tree) # new tree has pruned off the second split


# and this is how we might have included an early stopping rule
# rpart.control() is the function in which we include our early stopping rules
?rpart.control # for all of the possible options
control <- rpart.control(minbucket = 2) # we define the minimum number of observations in a ter
# now lets fit a new decision tree with an early stopping rule via control
```

```
43  early.stopping.fit <- rpart(Fruit ~ Width + Height, method = "class",
44                              data = fruit.data, minsplit = 2, control = control)
45  rpart.plot(early.stopping.fit) # it worked, a less deep tree was created with one terminal node
46
47
```

decision-tree-example.r hosted with ❤ by GitHub view raw

Well done on making it all the way to the end, follow me on twitter for a heads up on future posts. *All visualisations are my own.*

Machine Learning | Decision Tree | Tutorial | Classification | Data Science

Following

Written by Alan Jeffares

173 Followers · Writer for Towards Data Science

Machine Learning PhD student in University of Cambridge

More from Alan Jeffares and Towards Data Science